

Actividad 3. Aprendizaje Supervisado con Meta AI.

Prompt 1. ¿Cómo funciona matemáticamente el algoritmo K-Nearest Neighbors? Explica el papel de la distancia euclidiana, cómo impacta la elección del valor de K en el rendimiento del modelo (sobreajuste vs. subajuste), y por qué la normalización de los datos es un paso obligatorio antes de aplicar KNN.

KNN es un algoritmo de aprendizaje perezoso, o *lazy learning*. No "entrena" nada: solo memoriza todo el dataset. Cuando llega un punto nuevo para predecir, hace 3 pasos:

1. **Calcula la distancia** entre el punto nuevo y todos los puntos del dataset de entrenamiento
2. **Selecciona los K vecinos más cercanos** - los K puntos con menor distancia
3. **Vota o promedia**: Para clasificación, hace voto mayoritario de las clases de esos K vecinos. Para regresión, promedia los valores

No hay función matemática que se optimice. La "decisión" es 100% geométrica.

1. El papel de la distancia euclidiana

La distancia es el corazón de KNN. Define qué significa "cercano". La más común es la euclidiana:

Para dos puntos

$$p = (p_1, p_2, \dots, p_n)$$

y

$$q = (q_1, q_2, \dots, q_n)$$

en

n

dimensiones:

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Ejemplo en 2D: Si tienes edad e ingreso: punto A = [25 años, \$30k] y punto B = [30 años, \$80k]

$$d(A, B) = \sqrt{(25 - 30)^2 + (30000 - 80000)^2} = \sqrt{25 + 2,500,000,000} \approx 50000$$

¿Ves el problema? El ingreso domina completamente la distancia porque sus números son miles de veces más grandes. Por

Otras distancias que puedes usar: Manhattan

, Minkowski, Coseno. Pero euclidiana es default porque asume espacio euclidiano donde "línea recta" tiene sentido.

2. Cómo impacta K: Sobreajuste vs Subajuste

K controla la complejidad del modelo. Es el ejemplo más limpio de *bias-variance tradeoff*.

Valor de K	Frontera de decisión	Bias	Variance	Problema
K = 1	Extremadamente irregular, sigue cada punto de entrenamiento	Muy bajo	Muy alto	Sobreajuste / Overfitting. El modelo memoriza ruido y outliers. 1 solo vecino ruidoso te cambia la predicción
K pequeño, ej 3-5	Algo irregular pero suaviza ruido	Bajo	Alto	Buen balance si tienes muchos datos. Sensible a outliers locales
K grande, ej 50+	Muy suave, casi lineal	Alto	Bajo	Subajuste / Underfitting. Ignora patrones locales. Si K = N, siempre predices la clase mayoritaria del dataset
K = N	Frontera plana	Máximo	Mínimo	Clasificador tonto: siempre predice la moda

Regla práctica:

1. Empieza con $\sqrt{N}$ donde N = número de muestras de entrenamiento. Es heurística, no ley.

$$K = \sqrt{N}$$

2. Usa número impar para clasificación binaria y evitar empates.
3. Siempre valida K con cross-validation. Graficas "K vs Accuracy" y buscas el codo donde el error de validación es mínimo.

Intuición visual: K=1 dibuja islas alrededor de cada punto. K muy grande dibuja un país entero y borra las fronteras regionales.

3. ¿Por qué la normalización es obligatoria en KNN?

Porque KNN es 100% dependiente de distancias, y la distancia euclidiana no es invariante a la escala.

El problema sin normalizar:

Imagina 2 features: Edad 20-70 años, Salario \$20k-\$200k.

Punto a clasificar: [30 años, \$25k]

Vecino 1: [31 años, \$150k] →

$$d^2 = 1^2 + 125000^2$$

Vecino 2: [60 años, \$26k] →

$$d^2 = 30^2 + 1000^2$$

Aunque Vecino 2 tiene edad muy distinta, la distancia la domina el salario. KNN dirá que Vecino 2 es más cercano, cuando intuitivamente el 1 está más cerca en edad. El feature con escala mayor secuestra el algoritmo.

Qué hace la normalización:

Pone todas las features en la misma escala para que aporten equitativamente a la distancia.

Dos métodos estándar:

1. **Min-Max Scaling** → \$\$

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

1. **Standardization / Z-score** → media 0, desviación 1

$$x_{std} = \frac{x - \mu}{\sigma}$$

[0][1]

Después de normalizar, 1 año de diferencia pesa lo mismo que \$10k de diferencia, asumiendo que sus varianzas son similares.

Importante: Ajusta el `StandardScaler` solo con datos de entrenamiento, y usa esa misma transformación para test. Si normalizas todo junto tienes *data leakage*.

Excepción: Si todas tus features ya están en la misma escala natural, ej píxeles 0-255 en una imagen, puedes saltarte la normalización. Pero para datos tabulares mixtos es obligatoria.

Resumen rápido:

1. KNN decide por proximidad geométrica usando distancia euclidiana
2. K chico = modelo complejo que sobreajusta. K grande = modelo simple que subajusta
3. Sin normalizar, la feature con números más grandes decide todo y rompes el algoritmo

Conclusiones: El algoritmo de K vecinos cercanos memoriza los datos del arreglo para posteriormente, dado un input, buscar el vecino mas cercano (dato del arreglo) en características. Comparando además con los demás vecinos para hacer una deducción lo mas cercana posible a la caracterización real del input.

Prompt 2. ¿Qué miden exactamente la precisión, el recall (sensibilidad) y el F1-score en un problema de clasificación? ¿Por qué en un diagnóstico médico como

la detección de tumores malignos es preferible optimizar el recall sobre la precisión, incluso si esto aumenta los falsos positivos?

¿Qué miden precisión, recall y F1-score?

En clasificación binaria, todo se reduce a 4 conteos básicos:

≡ Métrica	Aa Fórmula	≡ Pregunta que responde	≡ Intuición	📎 Archivos
Precisión	Sin título	De todos los casos que dije que son positivos, ¿cuántos sí lo eran?	Mide qué tan "limpio" es tu detector. Pocos falsos positivos = precisión alta	$\frac{VP}{VP+FP}$
	Sin título			
Recall / Sensibilidad	Sin título	De todos los casos que realmente son positivos, ¿cuántos logré encontrar?	Mide qué tan "completo" es tu detector. Pocos falsos negativos = recall alto	$\frac{VP}{VP+FN}$
F1-score	Sin título	Balance entre precisión y recall	Media armónica. Castiga mucho si una de las dos es baja	$\frac{2 \cdot VP}{2 \cdot VP + FP + FN}$

VP = Verdaderos Positivos: detectaste tumor y sí había tumor

FP = Falsos Positivos: dijiste "tumor" pero era benigno/sano

FN = Falsos Negativos: dijiste "sano" pero sí había tumor

¿Por qué priorizar recall en detección de tumores malignos?

En medicina, no todos los errores cuestan lo mismo. La matriz de costos se ve así:

1. Falso Negativo: decirle "estás sano" a alguien con cáncer

- **Consecuencia:** El tumor sigue creciendo sin tratamiento. Para cuando dé síntomas, puede ser etapa 3 o 4.
- **Costo:** Vidas humanas, tratamientos más agresivos, peor pronóstico. Un FN es catastrófico.

2. Falso Positivo: decirle "posible tumor" a alguien sano

- **Consecuencia:** Ansiedad + biopsia/MRI extra para confirmar.
- **Costo:** Estrés, dinero y tiempo. Molesto, pero reversible. El paciente no muere por esto.

Por eso se optimiza recall: Prefieres atrapar al 99% de los tumores reales, aunque eso implique que de cada 100 alarmas, 30 sean falsas.

Es el principio de "mejor prevenir que lamentar". Un test de screening como mamografía o PSA está diseñado para ser la red amplia. La precisión se recupera después con pruebas confirmatorias más caras y específicas.

El tradeoff con un ejemplo

Imagina 1000 pacientes, 50 tienen tumor maligno:

Modelo A: Alta precisión, recall bajo

Detecta 30 casos. 28 son tumores reales, 2 son FP

Precisión = $28/30 = 93\%$ | Recall = $28/50 = 56\%$

Problema: 22 personas con cáncer se van a casa sin saberlo

Modelo B: Alto recall, precisión baja

Detecta 80 casos. 48 son tumores reales, 32 son FP

Precisión = $48/80 = 60\%$ | Recall = $48/50 = 96\%$

Resultado: Solo 2 cánceres se te escapan, pero 32 personas sanas hacen biopsia extra

En diagnóstico médico de cáncer, el Modelo B es éticamente preferible. Los FP generan estudios extra. Los FN generan muertes evitables.

Importante: Esto aplica a *screening*. Una vez que pasas a diagnóstico definitivo o cirugía, ahí sí quieres altísima precisión porque el costo de un FP es operar a alguien sano.

Conclusiones: Para estas conclusiones recordemos las diferencias entre cada una:

- **Precisión:** Mide que tan limpia es la detección. Pocos falsos positivos → alta precisión.
- **Recall (Sensibilidad):** Mide que tan completo es tu detector. Pocos falsos negativos → recall alto.
- **F1-score (Balance entre Precisión y Recall):** Media armónica. Castiga mucho si una de las dos es baja.

Es importante buscar un balance entre la precisión y recall para asegurar un correcto comportamiento en la arquitectura de nuestro modelo. Esto además permite determinar la calidad de un modelo pues en función de los resultados es posible determinar la eficacia del modelo. Por último, es importante considerar estas métricas de clasificación pues hay modelos cuyos resultados son cruciales para la toma de decisiones en la industria, áreas médicas y científicas.

Prompt 3. ¿Cuáles son los principales desafíos técnicos que distinguen un problema de clasificación binaria de uno multiclase? ¿Cómo adaptan los algoritmos como KNN su funcionamiento para manejar múltiples clases, y qué estrategias existen para mejorar el rendimiento en clases con pocos ejemplos?

Va, vamos más al grano y con ejemplos que se sientan.

1. Binario vs Multiclase: ¿Por qué multiclase duele más?

Piensa en trazar líneas para separar grupos.

Caso binario: Perros vs Gatos

Solo trazas 1 línea. Si te equivocas, el perro cae del lado de gatos. Fin.

Función de pérdida: ¿Es gato? Sí/No. Probabilidad con sigmoid: 0.8 = gato.

Caso multiclase: Perro vs Gato vs Conejo vs Hurón

Ahora necesitas separar 4 regiones. Tienes 3 opciones principales:

1. **One-vs-Rest OvR:** Entrenas 4 modelos.

Modelo 1: ¿Es Perro o No-Perro?

Modelo 2: ¿Es Gato o No-Gato?

Problema: Si Modelo 1 dice 0.9 Perro y Modelo 2 dice 0.7 Gato... ¿qué haces? Se pisan.

2. **One-vs-One OvO:** Entrenas 6 modelos.

Perro vs Gato, Perro vs Conejo, Perro vs Hurón...

Luego votan. Si tienes 100 clases son 4950 modelos. Se vuelve intratable.

3. **Softmax directo:** 1 sola red que saca 4 scores y softmax los convierte en probabilidades que suman 1.

→ La red tiene que aprender a no confundir Gato con Hurón y Perro con Conejo al mismo tiempo. Más difícil.

El problema real: Ambigüedad

Binario: Un correo es spam o no.

Multiclase: Una foto puede tener un "perro-gato" por la raza esfinge. ¿Es Gato? ¿Es Perro? ¿Es "Otro"? La frontera real es borrosa y el modelo se confunde más. Por eso multiclase necesita más datos.

2. KNN en multiclase: Cómo vota exactamente

KNN no cambia de algoritmo. Solo cambia cómo cuenta votos.

Paso a paso con K=7, 3 clases: A, B, C

1. Llega un punto nuevo X.
2. Calculas distancia de X a todos los 10,000 puntos de entrenamiento.
3. Tomas los 7 más cercanos. Supón que salen: A, A, B, C, A, B, C
4. Cuentas: A=3, B=2, C=2 → Predices A.

Dos trucos que usa KNN para multiclase:

Truco 1: Votos ponderados por distancia

Voto = $1 / \text{distancia}$. Si un vecino clase C está a distancia 0.1 y uno clase A está a 2.0:

C aporta $1/0.1 = 10$ votos.

A aporta $1/2.0 = 0.5$ votos.

Así evitas que un vecino lejano empate con uno pegadito. En sklearn: `weights='distance'`.

Truco 2: El valor de K importa más

Binario con K=3: necesitas que 2/3 vecinos sean de tu clase.

Multiclase con 10 clases y K=3: pueden salir 3 vecinos de 3 clases distintas. Empate triple.

Por eso en multiclase se usa K más grande: K=11, 15, 21. Da chance a que una clase domine.

Limitación dura de KNN: Si clase "Hurón" tiene solo 4 ejemplos en todo el dataset, es casi imposible que 5 vecinos sean Hurón. KNN jamás la va a predecir a menos que bajes K a 1, y entonces sobreajustas horrible. Ahí entran las estrategias para clases chicas.

3. Clases con pocos ejemplos: Qué hacer cuando "Hurón" solo tiene 5 fotos

Este es el problema de "long-tail". 80% de tus datos son Perro y Gato. 20% se reparte entre 50 animales raros.

Nivel 1: Arreglar los datos

Estrategia	Cómo funciona	Cuándo usarla con KNN
SMOTE	Toma un Hurón real, busca sus 5 vecinos Hurón más cercanos, y crea un Hurón sintético en la línea entre ellos.	Siempre. KNN depende de densidad local. SMOTE rellena huecos.
Class weights	Le dices al modelo: "Si te equivocas con Hurón, te castigo 20x más que si te equivocas con Perro"	KNN no tiene loss, pero sí <code>weights</code> . Usa <code>KNeighborsClassifier</code> y luego calibra con <code>CalibratedClassifierCV</code> .
Data augmentation	Foto de Hurón → la rotas, la oscureces, le pones ruido. Ahora tienes 5 → 50 Hurones.	Lo mejor si son imágenes/audio/texto. KNN ama más datos reales.

Nivel 2: Cambiar cómo decides

Imagina que tu modelo dice: `Perro: 0.45, Gato: 0.40, Hurón: 0.15`

Argmax diría "Perro". Pero sabes que Hurón es rarísimo y si el modelo le da 0.15 ya es mucho.

Ajuste de umbral: Dices "Si $P(\text{Hurón}) > 0.10$, predice Hurón aunque no sea la más alta". Estás moviendo la frontera a favor de la clase chica. Se calibra con la curva Precision-Recall.

Nivel 3: Cambiar el paradigma - Few-shot learning

Si solo tienes 5 Hurones, KNN normal muere. Mejor:

1. Agarras una ResNet preentrenada en ImageNet. Ya sabe qué es un ojo, pelo, bigotes.
2. Pasas tus 5 Hurones por la ResNet y sacas un vector de 2048 números por cada uno. Eso es un "embedding".
3. Sacas el promedio de esos 5 vectores. Ese promedio es el "prototipo de Hurón".
4. Para un punto nuevo, calculas distancia a cada prototipo. El más cercano gana.

Esto se llama **Prototypical Networks** y es KNN con $K=1$ pero sobre embeddings muy buenos. Con 5 ejemplos ya compites.

Regla de oro para medir: Olvida Accuracy. Si 99% de tus datos son Perro, un modelo tonto que siempre dice "Perro" tiene 99% accuracy y 0 utilidad.

Mira la **matriz de confusión** y el **Macro-F1**. Macro-F1 promedia el F1 de cada clase, así Hurón pesa lo mismo que Perro.

Ejemplo rápido:

Clase A: 1000 ejemplos. Predices bien 990. Recall = 99%

Clase B: 10 ejemplos. Predices bien 0. Recall = 0%

Accuracy = $990/1010 = 98\%$ → Se ve genial

Macro-F1 = promedio(99%, 0%) = 49.5% → Te dice la verdad: no sirves para la clase B

Conclusiones: Clasificar datos por clases o binariamente permite un manejo administrativo de la información, permitiendo separarlos de acuerdo a nuestro interés. Es importante porque permite tener una mejor higiene de los datos a medida que se analiza el dataset.